New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 15 — Módulo 4:

Autorización por Roles, Guards e Interceptores



Objetivo del módulo: implementar control de acceso granular en frontend y backend, configurando interceptores HTTP que inyectan el token Bearer, Guards funcionales para proteger rutas y directivas para acondicionar la interfaz según roles.



New Horizons®
Centro de Capacitación

Agenda

- Autorización basada en roles y claims en ASP.NET Core: Atributos y Políticas
- Interceptor HTTP funcional para la inyección automática de la cabecera Bearer Token
- Manejo de respuestas 401 Unauthorized y 403 Forbidden en el cliente
- Protección de rutas con Guards funcionales modernos (CanActivateFn) en Angular 18
- Acondicionamiento de vistas y botones de acción según el rol del usuario autenticado



New Horizons®
Centro de Capacitación

Sesión 1:

Autorización Basada en Roles, Permisos y Guards / Interceptores

Visión general de la sesión

New Horizons
Centro de Capacitación

Imagina el sistema como un edificio corporativo. La API en C# es la bóveda, Angular es el lobby. Hoy instalamos la seguridad de ambos: quién entra, a qué piso sube y qué puertas puede abrir.

Backend blindado

[Authorize], roles, claims y políticas en ASP.NET Core.

Piénsalo así: *La bóveda: es la única que decide de verdad quién abre cada caja.*

Token automático

Un interceptor añade el Bearer a cada petición HttpClient.

Piénsalo así: *La oficina de correos: sella cada carta que sale, sin que lo pidas.*

Guards y UI por rol

Rutas protegidas y botones que aparecen según el rol.

Piénsalo así: *El vigilante del lobby y el ascensor que solo muestra tus pisos.*

Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

Los 5 pisos de seguridad de hoy: cada pilar resuelve una pregunta distinta del control de acceso.

1 

Roles y políticas

Autorización basada en roles y claims en ASP.NET Core: atributos y políticas.

¿Quién puede hacer qué?

≈ El reglamento

2 

Interceptor

Inyección automática de la cabecera Bearer Token.

¿Cómo viaja la credencial?

≈ El sello postal

3 

401 / 403

Manejo de respuestas Unauthorized y Forbidden en el cliente.

¿Qué hago si me rechazan?

≈ Dos portazos distintos

4 

Guards

Protección de rutas con Guards funcionales (CanActivateFn) en Angular 18.

¿Puede entrar a esta pantalla?

≈ El vigilante

5 

UI por rol

Vistas y botones de acción según el rol del usuario autenticado.

¿Qué botones ve?

≈ El ascensor

Políticas de autorización en ASP.NET Core

New Horizons
Centro de Capacitación

Program.cs

```
// Program.cs: autenticación + reglas de autorización
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(); // configurado en la Sesión 13

1  builder.Services.AddAuthorization(options =>
   {
       options.AddPolicy("PuedeAprobar", p =>
2          p.RequireRole("Aprobador", "Administrador"));

       options.AddPolicy("MontoAlto", p =>
3          p.RequireClaim("nivelAprobacion", "2", "3"));
   });

   var app = builder.Build();
4  app.UseAuthentication();    // ¿quién eres?
   app.UseAuthorization();     // ¿qué puedes hacer?
```

El reglamento del edificio

Las políticas son reglas con nombre. El controlador solo dice “aplica la regla PuedeAprobar”.

1 AddAuthorization

Registra el reglamento completo en un solo lugar.

2 RequireRole

Regla por cargo: Aprobador o Administrador.

3 RequireClaim

Regla por dato del token: nivel de aprobación 2 o 3.

4 El orden importa

Primero se identifica (Authentication), luego se autoriza.

Práctica: Interceptor HTTP para el Bearer Token

New Horizons®
Centro de Capacitación

Hoy montamos la oficina de correos de Angular: cada petición que sale recibe su sello (el token) sin que ningún servicio tenga que acordarse de ponerlo.

1 Crea el archivo

En src/app/interceptors:

```
ng g interceptor
  interceptors/auth
```

2 Escribe la función

Un HttpInterceptorFn que clona la petición:

```
req.clone({ setHeaders })
```

3 Regístralo

En app.config.ts:

```
withInterceptors(
  [authInterceptor])
```

4 Verifica

DevTools → Network → Headers:

```
Authorization: Bearer eyJ…
```

Clonación inmutable: un HttpRequest es como un sobre ya cerrado. No se modifica; se hace una copia con el sello usando `req.clone()`.

Roles y claims: lo que dice tu token

payload del JWT (decodificado en jwt.io)

```
{
  "sub": "1024",
  "name": "Ana Torres",
  "email": "ana.torres@empresa.pe",
1 "role": ["Aprobador"],
2 "nivelAprobacion": "2",
  "centroCosto": "TI-01",
3 "exp": 1767225600
}
```

1 role

El cargo: lo lee [Authorize(Roles = ...)].

2 claims personalizados

Datos extra: nivel, centro de costo.

3 exp

Fecha de vencimiento: pasada esa hora, la API responde 401.

El fotocheck de la empresa

El rol es el cargo impreso; los claims son los demás datos de la tarjeta: área, nivel, vencimiento.

Rol *¿Qué cargo tienes?*

```
RequireRole("Aprobador")
```

Claim *¿Qué dato tienes?*

```
RequireClaim("nivelAprobacion")
```

⚠ El JWT está **firmado, no cifrado**: cualquiera puede leerlo. Nunca pongas sueldos ni datos sensibles en un claim.

[Authorize] en los controladores

New Horizons®
Centro de Capacitación

SolicitudesController.cs

```
[ApiController]
[Route("api/[controller]")]
1 [Authorize]
public class SolicitudesController : ControllerBase
{
    [HttpGet]
    public IActionResult Listar() => Ok(/* ... */);

    [HttpPut("{id}/aprobar")]
2   [Authorize(Policy = "PuedeAprobar")]
    public IActionResult Aprobar(int id) => NoContent();

    [HttpDelete("{id}")]
3   [Authorize(Roles = "Administrador")]
    public IActionResult Eliminar(int id) => NoContent();

    [HttpGet("publico/estado")]
4   [AllowAnonymous]
    public IActionResult Estado() => Ok("API en línea");
}
```



Carteles en cada puerta

"Solo personal", "Solo aprobadores", "Solo gerencia", "Entrada libre".

1 **[Authorize] en la clase**

Todo el piso exige fotocheck: sin token, 401.

2 **Policy = "PuedeAprobar"**

Aplica la regla con nombre definida en Program.cs.

3 **Roles = "Administrador"**

Cargo explícito; si no lo tienes, 403.

4 **[AllowAnonymous]**

La recepción: abierta a todos, incluso sin login.

Interceptor funcional: authInterceptor

New Horizons®
Centro de Capacitación

auth.interceptor.ts

```ts
import { HttpInterceptorFn } from "@angular/common/http";
import { inject } from "@angular/core";
import { AuthService } from "../services/auth.service";

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).obtenerToken();

  if (!token || req.url.includes("/auth/login")) {
    return next(req);
  }

  const reqConToken = req.clone({
    setHeaders: { Authorization: `Bearer ${token}` }
  });
  return next(reqConToken);
};
```

La oficina de correos

Cada carta (request) pasa por la ventanilla y sale con su sello (Bearer). Nadie lo pega a mano.

1 — Una función, no una clase

HttpInterceptorFn: más simple y fácil de testear.

2 — inject() dentro de la función

Toma el token del AuthService de la Sesión 14.

3 — No todo se sella

Sin token o en el login, la carta sale tal cual.

4 — clone() + setHeaders

La petición original es inmutable: se envía una copia.

Registro de interceptores en app.config.ts

New Horizons
Centro de Capacitación

app.config.ts

```ts
import { ApplicationConfig } from "@angular/core";
import { provideRouter } from "@angular/router";
import { provideHttpClient, withInterceptors }
  from "@angular/common/http";
import { routes } from "./app.routes";
import { authInterceptor } from "./interceptors/auth.interceptor";
import { errorInterceptor } from "./interceptors/error.interceptor";

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(                                  // 1
      withInterceptors([authInterceptor, errorInterceptor])  // 2
    )
  ]
};
```



Una línea de ensamblaje

El orden del arreglo es el orden de las estaciones: la petición pasa por auth y luego por error.

La misma petición, antes y después

Antes

```
GET /api/solicitudes

(sin cabecera Authorization)
```

Después

```
GET /api/solicitudes

Authorization: Bearer eyJhbGciOi…
```

1. **provideHttpClient**

   Reemplaza a HttpClientModule en apps standalone.

2. **withInterceptors([...])**

   La respuesta vuelve en orden inverso.

401 vs. 403: dos rechazos distintos

New Horizons
Centro de Capacitación

En la puerta de la discoteca hay dos formas de quedarse afuera: no traer documento o traerlo y no estar en la lista. Para la API son el 401 y el 403.

401 Unauthorized

¿Quién eres?

Causa

Sin token, token vencido o firma inválida.

Analogía

Llegas sin DNI: no saben quién eres.

Qué hace el cliente: Cerrar sesión y llevar a /login con returnUrl.

403 Forbidden

Sé quién eres, pero no puedes pasar.

Causa

Token válido, pero sin el rol o la política.

Analogía

Tienes DNI, pero no estás en la lista VIP.

Qué hace el cliente: Mostrar /acceso-denegado. NO cerrar sesión.

Interceptor de errores: manejo de 401 y 403

New Horizons®
Centro de Capacitación

error.interceptor.ts

```
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const auth = inject(AuthService);

1   return next(req).pipe(
      catchError((error: HttpErrorResponse) => {
2       if (error.status === 401) {
          auth.cerrarSesion();
          router.navigate(["/login"],
            { queryParams: { returnUrl: router.url } });
3       } else if (error.status === 403) {
          router.navigate(["/acceso-denegado"]);
        }
4       return throwError(() => error);
      })
  );
};
```



El recepcionista que te reorienta

Sin credencial, te manda a registrarte. Si ese piso no es tuyo, te indica la salida correcta.

1 pipe + catchError

Escucha la respuesta de regreso, no la petición.

2 401 → login con returnUrl

Tras volver a ingresar, regresa a donde estaba.

3 403 → acceso denegado

La sesión sigue viva: solo le falta permiso.

4 throwError

Relanza el error para que el componente también se entere.

Guards funcionales con CanActivateFn

New Horizons®
Centro de Capacitación

auth.guard.ts

```ts
import { inject } from "@angular/core";
import { CanActivateFn, Router } from "@angular/router";
import { AuthService } from "../services/auth.service";

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  if (auth.estaAutenticado()) {
    return true;
  }
  return router.createUrlTree(["/login"],
    { queryParams: { returnUrl: state.url } });
};
```

1
2
3



El vigilante del edificio

Revisa tu fotocheck antes de dejarte subir. Escribir la URL a mano no lo esquiva: sigue en la puerta.

1 CanActivateFn

Una función que se ejecuta antes de cargar la ruta.

2 return true

Fotocheck válido: puede pasar.

3 createUrlTree

Redirige de forma limpia, mejor que navigate + false.

`/solicitudes` → `authGuard` → ✅ o ↩ /login

Guard por roles con parámetros

New Horizons®
Centro de Capacitación

role.guard.ts

```ts
export const roleGuard =
  (rolesPermitidos: string[]): CanActivateFn => () => {
    const auth = inject(AuthService);
    const router = inject(Router);
    const tieneRol = rolesPermitidos.some(r => auth.tieneRol(r));
    return tieneRol
      ? true
      : router.createUrlTree(["/acceso-denegado"]);
  };

// Uso: una fábrica, muchos vigilantes
{ path: "aprobaciones", canActivate: [roleGuard(["Aprobador"])] }
```

Tarjeta de acceso por piso

Una sola fábrica de vigilantes: a cada piso le dices qué cargos pueden subir.

Piso 3 · Administración	Administrador
Piso 2 · Aprobaciones	Aprobador, Administrador
Piso 1 · Mis solicitudes	Cualquier usuario autenticado
Lobby · Login	Público

1 Función que devuelve un guard

Recibe los roles y fabrica un CanActivateFn.

Rutas protegidas en app.routes.ts

New Horizons
Centro de Capacitación

app.routes.ts

```
export const routes: Routes = [
  { path: "login", component: LoginComponent },
  { path: "acceso-denegado", component: AccesoDenegadoComponent },
  {
    path: "",
1   canActivate: [authGuard],
    children: [
      { path: "solicitudes", component: MisSolicitudesComponent },
      { path: "aprobaciones", component: BandejaAprobacionComponent,
2       canActivate: [roleGuard(["Aprobador", "Administrador"])] },
      { path: "admin", component: AdminComponent,
3       canActivate: [roleGuard(["Administrador"])] }
    ]
  },
4 { path: "**", redirectTo: "solicitudes" }
];
```



El directorio del lobby

Muestra cada piso y quién puede subir. El vigilante general cuida la puerta de todo el edificio.

¿Quién entra a cada ruta?

`/login`	🌐 Todos
`/solicitudes`	🔑 Autenticados
`/aprobaciones`	🕵️ Aprobador, Admin
`/admin`	👑 Administrador

💡 **Guard en el padre:** protege a todos los hijos de una sola vez.

Directiva *appTieneRol para la interfaz

New Horizons
Centro de Capacitación

tiene-rol.directive.ts

```
1  @Directive({ selector: "[appTieneRol]", standalone: true })
   export class TieneRolDirective {
     private tpl = inject(TemplateRef);
     private vcr = inject(ViewContainerRef);
     private auth = inject(AuthService);

2    @Input() set appTieneRol(roles: string[]) {
       this.vcr.clear();
       if (roles.some(r => this.auth.tieneRol(r))) {
3        this.vcr.createEmbeddedView(this.tpl);
       }
     }
   }

   <!-- Uso en la bandeja -->
4  <button *appTieneRol="['Aprobador']"
           (click)="aprobar(s)">Aprobar</button>
```



El ascensor inteligente

Solo muestra los botones de los pisos a los que tu tarjeta da acceso.

1. **Directiva estructural**
   Como un @if, pero que pregunta por roles.
2. **@Input set**
   Se reevalúa cada vez que cambian los roles.
3. **createEmbeddedView**
   Pinta el elemento solo si hay coincidencia.
4. **Un atributo en el HTML**
   El componente no necesita lógica de roles.

⚠ **Ocultar no es proteger:** el endpoint sigue con [Authorize].

Una pantalla, tres roles

New Horizons®
Centro de Capacitación

🎭 **Mismo teatro, distinta entrada:** platea, palco o backstage. La bandeja de solicitudes es una sola, pero cada rol ve acciones distintas.

👤 Solicitante

Bandeja de solicitudes

+ Nueva solicitud

SOL-0412	Taxi aeropuerto	Ver
SOL-0415	Hotel 2 noches	Ver
SOL-0419	Almuerzo cliente	Ver

👨‍⚖️ Aprobador

Bandeja de solicitudes

SOL-0412	Taxi aeropuerto	Aprobar	Rechazar
SOL-0415	Hotel 2 noches	Aprobar	Rechazar
SOL-0419	Almuerzo cliente	Aprobar	Rechazar

👑 Administrador

Bandeja de solicitudes

⚙ Administración

SOL-0412	Taxi aeropuerto	Aprobar	Rechazar	🗑
SOL-0415	Hotel 2 noches	Aprobar	Rechazar	🗑
SOL-0419	Almuerzo cliente	Aprobar	Rechazar	🗑

Defensa en profundidad

New Horizons
Centro de Capacitación

Un castillo no confía en un solo muro. Si una capa falla o la saltan, la siguiente sigue en pie. La última, la del backend, nunca se negocia.

1

UI

`*appTieneRol`

Cortina: oculta lo que no te toca.

Si la saltan:

Alguien curioso ve el botón… → el Guard igual lo detiene.

2

Guard

`roleGuard`

Puerta: bloquea la ruta.

Si la saltan:

Escribe la URL a mano… → la API responde 403.

3

Interceptor

`authInterceptor`

Mensajero: adjunta la credencial.

Si la saltan:

Token vencido… → 401 y vuelta al login.

4

API

`[Authorize]`

Bóveda: decide de verdad.

Si la saltan:

Llamada desde Postman… → rechazada igual.

Frontend es cortesía, backend es ley: las capas 1–3 mejoran la experiencia; la 4 garantiza la seguridad.

Flujo completo de una petición autorizada

New Horizons
Centro de Capacitación

La vida de un clic: como una carta certificada, pasa por varias ventanillas antes de llegar a destino.

1



Ana hace clic en “Aprobar”

El botón existe porque *appTieneRol la reconoce como Aprobador.

2



La ruta ya fue validada

roleGuard dejó entrar a /aprobaciones.

3



Se sella la petición

authInterceptor agrega Authorization: Bearer …

4



La API autentica

Valida firma y vencimiento del JWT.

5



La API autoriza

La política PuedeAprobar revisa el rol.

6



Llega la respuesta

204 ✅ · 401 → login · 403 → acceso denegado

Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Usar variables de entorno seguras en desarrollo y producción
Desacoplar la UI de las políticas de seguridad del backend	Derivar los roles de la UI de los mismos claims del token
Confiar solo en ocultar botones	Proteger cada endpoint con [Authorize] o una política
Modificar la request original en el interceptor	Usar req.clone(): HttpRequest es inmutable
Cerrar sesión ante un 403	Ante 403, mostrar acceso denegado; cerrar sesión solo ante 401

Checklist de seguridad antes de producción

New Horizons
Centro de Capacitación



La ronda del vigilante nocturno: antes de cerrar el edificio, revisa cada puerta una por una.

✓ **Cada endpoint sensible tiene [Authorize] o una política.**	→ *Ninguna puerta queda abierta por olvido.*
✓ **UseAuthentication va antes de UseAuthorization.**	→ *El orden del pipeline importa.*
✓ **El interceptor no envía el token a dominios externos.**	→ *Tu credencial no viaja a terceros.*
✓ **401 → login con returnUrl; 403 → acceso denegado.**	→ *El usuario sabe qué pasó y qué hacer.*
✓ **Toda ruta privada tiene authGuard o roleGuard.**	→ *No se entra tecleando la URL.*
✓ ***appTieneRol usa los mismos roles que el backend.**	→ *UI coherente con las políticas.*

Resumen de la Sesión 15

New Horizons®
Centro de Capacitación

Roles y políticas

[Authorize] y las políticas deciden en el servidor quién ejecuta cada endpoint.

≈ El reglamento

Interceptor HTTP

Los interceptores funcionales centralizan la inyección de seguridad en una sola función pura.

≈ El sello postal

401 y 403

Permiten redirigir a Login o a una pantalla de acceso denegado según el caso.

≈ Dos portazos

Guards

Protegen la navegación en el cliente pero nunca sustituyen la seguridad del backend.

≈ El vigilante

UI por rol

El control de acceso por roles mantiene la interfaz limpia mostrando solo acciones autorizadas.

≈ El ascensor

Full Stack

La solución queda blindada de extremo a extremo.

≈ El castillo

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cómo evitamos tener que añadir manualmente el token en cada llamada HttpClient?
2. ¿Qué diferencia existe entre un código HTTP 401 (Unauthorized) y un 403 (Forbidden)?
3. ¿Cómo impide un Guard que un usuario acceda escribiendo directamente la URL en el navegador?
4. ¿Por qué es indispensable validar la autorización en el backend aunque el frontend la oculte?
5. ¿Cómo estructuramos un Guard modular que reciba los roles permitidos por parámetro?

New Horizons
Centro de Capacitación

Conclusiones



Un solo punto de sellado

Los interceptores funcionales centralizan la inyección de seguridad en una sola función pura.



El vigilante no es la bóveda

Los Guards protegen la navegación en el cliente pero nunca sustituyen la seguridad del backend.



Dos rechazos, dos caminos

La combinación de 401 y 403 permite redirigir a Login o a una pantalla de acceso denegado según el caso.



Solo los botones que tocan

El control de acceso por roles mantiene la interfaz limpia mostrando solo acciones autorizadas.



Defensa en profundidad

La solución Full Stack queda completamente blindada de extremo a extremo.

GRACIAS